CS 300 Project One
Jessica Soler

CLIENT

OVERVIEW

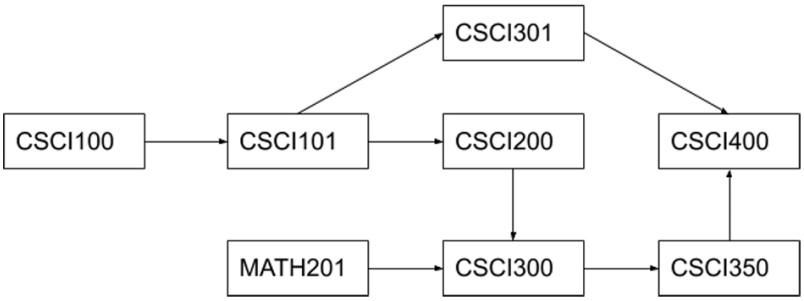
ABC University (ABCU) is looking for software to help computer science advisors access course information for students. The software needs to load course data from a CSV file, display all courses in alphanumeric order, and allow users to search and view a specific course with its prerequisite(s).

To determine the most effective solution, three data structures are evaluated: vector, hash table, binary search tree. Each structure is evaluated based on its ability to store, retrieve, and display course data efficiently. The final recommendation is based on runtime performance, memory usage, and how effective each structure solves the program requirements.

The Course data is as follows:

CSCI100,Introduction to Computer Science
CSCI101,Introduction to Programming in C++,CSCI100
CSCI200,Data Structures,CSCI101
MATH201,Discrete Mathematics
CSCI300,Introduction to Algorithms,CSCI200,MATH201
CSCI301,Advanced Programming in C++,CSCI101
CSCI350,Operating Systems,CSCI300
CSCI400,Large Software Development,CSCI301,CSCI350

The Course data visualized as a flowchart:



PSEUDOCODE

The following pseudocode outlines the structure for the ABCU Advising System. It begins with the course object and menu logic which are shared across all implementations. Following, three separate data structure implementations are presented: vector, hash table, and binary search tree. Each implementation includes its own functions for loading course data, searching for a course, and printing course information. They represent three potential approaches to solving the same problem and will be further evaluated to determine the most efficient solution.

COURSE OBJECT

```
/**
* custom data structure object that holds data parsed from the input file
**/
STRUCT Course:
    String courseNumber
    String title
    Vector<String> prerequisites          // Stores IDs of required courses
```

MENU LOGIC

```
/**
* handles user interaction
**/
FUNCTION main():
    SET choice = 0
    CONST MENU_OPTIONS = [1, 2, 3, 9]
    // Abstract container for course data depending on the chosen structure
    // Vector<Course>, HashTable<String, Course>, or Node (root of BST)
    DECLARE dataStructure

    WHILE choice != 9:
        PRINT "1.  Load Courses"
        PRINT "2.  Print All Courses"
        PRINT "3.  Print One Course"
        PRINT "9.  Exit"

        INPUT choice
        // input validation
        IF choice is not numeric OR choice NOT IN MENU_OPTIONS:
            PRINT "Error: Invalid Selection.  Please enter 1, 2, 3, or 9."
            CONTINUE

        SWTICH choice:
            CASE 1:
                SET filename = "CS 300 ABCU_Advising_Program_Input.csv"
                SET dataStructure = loadCourses(fileName)
            CASE 2:
```

```
        IF dataStructure is null:
            PRINT "Error: Please load data."
        ELSE:
            printCourseList(dataStructure)
    CASE 3:
        IF dataStructure is null:
            PRINT "Error: Please load data."
        ELSE:
                PRINT "Enter course number (ex: CSCI100)"
                INPUT courseNumber
                // input validation
                IF courseNumber is empty:
                    PRINT "ERROR: Course number cannot be blank."
                ELSE:
                    CONVERT courseNumber TO uppercase
                    searchPrintCourse(dataStructure, courseNumber)
    CASE 9:
        PRINT "Exiting the program.  Goodbye!"
```

VECTOR IMPLEMENTATION

```
/**
* opens and validates data input file
* creates allCourses vector to store the course data
* read the data from the file, parse each line, check for data format errors
* creates a course object, stores courseNumber, title, and prerequisites
* adds validated course object to allCourses vector
**/
FUNCTION loadCourses(fileName):
    OPEN fileName
    IF file is not open:
        PRINT "Error: could not open file"
        RETURN null

    CREATE Vector<Course> allCourses
    SET fileIsValid = TRUE

    // First Pass: Load all courses and validate data format and structure
    WHILE NOT end of file:
        READ line
        PARSE line into tokens using commas as delimiter

        // Validation: Ensure at least two parameters (courseNumber and title)
        IF tokens.size < 2:
            PRINT "Error: Missing course number or course title"
```

```
            SET fileIsValid = FALSE
            CONTINUE

        CREATE newCourse object
        newCourse.courseNumber = tokens[0]
        newCourse.title = tokens[1]

        // add prerequisites if they exist
        FOR EACH token starting from index 2:
            ADD token to newCourse.prerequisites

        ADD newCourse to allCourses vector
    CLOSE file

    // Second Pass: Ensure prerequisites exist as a course
    FOR EACH course IN allCourses:
        FOR EACH prereqID IN course.prerequisites:
            SET isFound = FALSE
            FOR EACH checkCourse IN allCourses:
                IF checkCourse.courseNumber EQUALS prereqID:
                    SET isFound = TRUE
                    BREAK
            IF isFound is FALSE:
                PRINT "Error: Prerequisite " + prereqID + " does not exist"
                SET fileIsValid = FALSE

    IF fileIsValid is FALSE:
        PRINT "ERROR: Course data failed validation."
        RETURN null

    PRINT "Courses loaded successfully."
    RETURN allCourses

/**
* performs a linear search through the vector to locate a course
* and prints the associated prerequisite courses
**/
FUNCTION searchPrintCourse(Vector<Course> courses, String courseNumber):
    SET foundStatus = FALSE
    // Iterate through vector
    FOR EACH currentCourse IN courses:
        // Check if course matches the search key
```

```
        IF currentCourse.courseNumber EQUALS courseNumber:
            PRINT currentCourse.courseNumber + ": " + currentCourse.title

            // handle printing prerequisites
            IF currentCourse.prerequisites.size > 0:
                PRINT "Prerequisites: "
                FOR EACH prerequisiteID IN currentCourse.prerequisites:
                    SET prereqFound = FALSE
                    // search for the prerequisite course in the vector
                    FOR EACH prerequisiteCourse IN courses:
                        IF prerequisiteCourse.courseNumber EQUALS prerequisiteID:
                            PRINT prerequisiteCourse.courseNumber + ": " + prerequisiteCourse.title
                            SET prereqFound = TRUE
                            BREAK
                    // handle edge cases
                    IF prereqFound is FALSE:
                        PRINT "Error: " + prerequisiteID + " was not found"
            ELSE:
                PRINT "Prerequisites: None"

            SET foundStatus = TRUE
            BREAK

    // Handle a case where course is not found
    IF foundStatus is FALSE:
        PRINT "Course " + courseNumber + " not found"

/**
* sorts courses alphanumerically by course number
 * prints all courses
**/
FUNCTION printCourseList(Vector<Course> courses):
    SORT courses BY course.courseNumber ASCENDING

    FOR EACH course IN courses:
        PRINT course.courseNumber + ": " + course.title
```

HASH TABLE IMPLEMENTATION

```
/**
* opens and validates data input file
* creates courseTable hash table to store course data
```

```
* reads data from file, parses each line, checks for data format errors
* creates course object, stores courseNumber, title, prerequisites
* inserts validated course object into courseTable using courseNumber as key
**/
FUNCTION loadCourses(fileName):
    OPEN fileName
    IF file is not open:
        PRINT "Error: could not open file"
        RETURN null

    CREATE HashTable<String, Course> courseTable
    SET fileIsValid = TRUE

    // First Pass: Load all courses and validate basic structure
    WHILE NOT end of file:
        READ line
        PARSE line into tokens using commas as delimiter

        // Validation: Ensure at least two parameters (courseNumber and title)
        IF tokens.size < 2:
            PRINT "Error: Missing ID or title"
            SET fileIsValid = FALSE
            CONTINUE

        CREATE newCourse object
        newCourse.courseNumber = tokens[0]
        newCourse.title = tokens[1]

        // add prerequisites if they exist
        FOR EACH token starting from index 2:
            ADD token to newCourse.prerequisites

        // Store course in hash table using course number as key
        INSERT newCourse INTO courseTable USING newCourse.courseNumber AS key
    CLOSE file

    // Second Pass: Ensure prerequisites exist as a course
    FOR EACH key IN courseTable:
        SET course = courseTable[key]

        FOR EACH prereqID IN course.prerequisites:
            IF courseTable does NOT contain key prereqID:
                PRINT "Error: Prerequisite " + prereqID + " does not exist"
                SET fileIsValid = FALSE

    IF fileIsValid is FALSE:
        PRINT "ERROR: Course data failed validation."
        RETURN nul
```

```
    PRINT "Courses loaded successfully."
    RETURN courseTable

/**
* searches the hash table for a course
* and prints the associated prerequisites
**/
FUNCTION searchPrintCourse(HashTable<String, Course> courses, String searchID):
    // check if the course exists in the hash table
    IF courses contains key courseNumber:
        SET currentCourse = courses[courseNumber]
        PRINT currentCourse.courseNumber + ": " + currentCourse.title

        // handle printing prerequisites
        IF currentCourse.prerequisits.size > 0:
            PRINT "Prerequisites: "
            FOR EACH prerequisiteID IN currentCourse.prerequisites:
                IF courses contains key prerequisitesID:
                    SET prerequisiteCourse = courses[prerequisiteID]
                    PRINT prerequisiteCourse.courseNumber + ": " + prerequisiteCourse.title
                ELSE:
                    PRINT "Error: Course " + prerequisiteID + " not found."
        ELSE:
            PRINT "Prerequisites: None"
    ELSE:
        PRINT "Error: Course " + prerequisiteID + " not found."

/**
* copies courses into a temporary vector
* sorts courses alphanumerically by course number
* prints all courses
**/
FUNCITON printCourseList(HashTable<String, Course> courses):
    CREATE Vector<Course> sortedCourses

    FOR EACH key IN courses:
        ADD courses[key] TO sortedCourses

    SORT sortedCourses BY course.courseNumber ASCENDING

    FOR EACH course IN sortedCourses:
        PRINT course.courseNumber + ": " + course.title
```

BINARY SEARCH TREE IMPLEMENTATION

```
/**
* node structure for binary search tree
```

```
**/
STRUCT Node:
    Course course
    Node left
    Node right

/**
* inserts a course into the binary search tree
**/
FUNCTION insertCourse(root, newCourse):
    IF root is null:
        CREATE newNode
        newNode.course = newCourse
        newNode.left = null
        newNode.right = null
        return newNode

    IF newCourse.courseNumber LESS THAN root.course.courseNumber:
        root.left = insertCourse(root.left, newCourse)
    ELSE:
        root.right = insertCourse(root.right, newCourse)

    RETURN root

/**
* searches the binary search tree for a course
**/
FUNCTION searchTree(root, courseNumber):
    IF root is null:
        RETURN null

    IF root.course.courseNumber EQUALS courseNumber:
        RETURN root.course

    IF courseNumber LESS THAN root.course.courseNumber:
        RETURN searchTree(root.left, courseNumber)
    ELSE:
        RETURN searchTree(root.right, courseNumber)

/**
* opens and validates data input file
* creates binary search tree using root node
* stores course data in tree and validates prerequisites
**/
FUNCTION loadCourses(fileName):
    OPEN fileName
        IF file is not open:
            PRINT "Error: could not open file."
```

```
        RETURN null

    SET root = null
    CREATE Vector<Courses> allCourses
    SET fileIsValid = TRUE

    // First Pass: Load courses into tree
    WHILE NOT end of file:
        READ line
        PARSE line into tokens using comma as delimiter

        IF tokens.size < 2:
            PRINT: "Error: Missing course number or course title."
            SET fileIsValid = FALSE
            CONTINUE

        CREATE newCourse object
        newCourse.courseNumber = tokens[0]
        newCourse.title = tokens[1]

        FOR EACH token starting from index 2:
            ADD token to newCourse.prerequisites

        SET root = insertCourse(root, newCourse)
        ADD newCourse to allCourses vector
    CLOSE file

    // validate prerequisites using tree search
    FOR EACH course IN allCourses:
        FOR EACH prereqID IN course.prerequisites:
            SET foundCourse = searchTree(root, rereqID)

            IF foundCourse is null:
                PRINT "Error: Prerequisite " + prereqID + " does not exist."
                SET fileIsValid = FALSE

    IF fileIsValid is FALSE:
        PRINT "Error: course data failed validation."
        RETURN null
    PRINT "Courses loaded successfully."
    RETURN root

/**
* searches the tree and prints course and prerequisites
**/
FUNCTION searchPrintCourse(root, courseNumber):
    SET currentCourse = searchTree(root, courseNumber)
```

```
    IF currentCourse is null:
        PRINT "Course " + courseNumber + " not found"
        RETURN

    PRINT currentCourse.courseNumber + ": " + currentCourse.title

    IF currentCourse.prerequisites.size > 0:
        PRINT "Prerequisites: "
        FOR EACH prerequisiteID IN currentCourse.prerequisites:
            SET prerequisiteCourse = searchTree(root, prerequisiteID)

        IF prerequisiteCourse is not null:
            PRINT prerequisiteCourse.courseNumber + ": " + prerequisiteCourse.title
        ELSE:
            PRINT "Error: " + prerequisiteID + " was not found"
    ELSE:
        PRINT "Prerequisites: None"

/**
* performs in-order traversal to print courses in sorted order
**/
FUNCTION inOrderPrint(root):
    IF root is null:
        RETURN

    inOrderPrint(root.left)
    PRINT root.course.courseNumber + ": " + root.course.title
    inOrderPrint(root.right)

/**
* prints all courses in alphanumeric order
**/
FUNCTION printCourseList(root):
    inOrderPrint(root)
```

RUNTIME ANALYSIS

VECTOR RUNTIME ANALYSIS – WORST CASE			
CODE	**LINE COST**	**EXECUTIONS**	**TOTAL COST**
WHILE NOT end of file	1	n	n
READ line	1	n	n
PARSE line	1	n	n
CREATE newCourse obect	1	n	n
ADD newCourse to Vector	1	n	n
FOR EACH course IN allCourses	1	n	n
FOR EACH prereqID IN course	1	n	n
FOR EACH checkCourse IN allCourses	1	n	n^2
		Total Cost	**7n + n^2**
		Runtime	**O(n^2)**

HASH TABLE RUNTIME ANALYSIS – WORST CASE			
CODE	**LINE COST**	**EXECUTIONS**	**TOTAL COST**
WHILE NOT end of file	1	n	n
READ line	1	n	n
PARSE line	1	n	n
CREATE newCourse object	1	n	n
ADD newCourse to Vector	n	n	n^2
FOR EACH course IN table	1	n	n
FOR EACH prereqID IN course	1	n	n
HASH LOOKUP	n	n	n^2
		Total Cost	**6n + 2n^2**
		Runtime	**O(n^2)**

BINARY SEARCH TREE RUNTIME ANALYSIS – WORST CASE			
CODE	**LINE COST**	**EXECUTIONS**	**TOTAL COST**
WHILE NOT end of file	1	n	n
READ line	1	n	n
PARSE line	1	n	n
CREATE newCourse object	1	n	n
ADD newCourse to BST	n	n	n^2
FOR EACH course IN allCourses	1	n	n
FOR EACH prereqID IN course	1	n	n
TREE SEARCH	n	n	n^2
		Total Cost	**6n + 2n^2**
		Runtime	**O(n^2)**

RUNTIME SUMMARY

Let n represent the number of courses in the dataset. When evaluating for worst case conditions, the vector, hash table, and binary search tree implementations each have an overall runtime of $O(n^2)$. The vector reaches $O(n^2)$ because prerequisite validation requires a linear search through the entire vector for every course entry. The hash table reaches $O(n^2)$ due to collision and repeated linear searching through the chain. The hash table reaches $O(n^2)$ with an unbalanced tree that acts like a linked list, requiring linear search. Although their worst case runtimes are the same, the differences in how each structure reaches $O(n^2)$.

ADVANTAGES AND DISADVANTAGES

VECTOR

- Advantages: The vector is simple to implement and uses contiguous memory. Therefore, it is the most memory-efficient structure.

- Disadvantages: The vector has poor performance during prerequisite validation because it requires repeated linear searches through the entire dataset. This results in a worst case runtime of $O(n^2)$. It does not hold data in a sorted order, requiring a separate sorting method to display all courses.

HASH TABLE

- Advantages: The hash table provides efficient average case performance for insertion and lookup operations. It allows direct access to course data using a key, which improves performance compared to linear searches.

- Disadvantages: The hash table does not maintain data in sorted order. This requires an additional step to sort data before printing. In the worst case, collisions can degrade performance to $O(n^2)$. It also requires memory for empty buckets.

BINARY SEARCH TREE

- Advantages: The binary search tree does not require a separate sorting method because it is stored in a sorted order. It supports efficient searching and insertion when the tree is balanced.

- Disadvantages: The binary search tree is more complex to implement. It uses additional memory for node pointers. In the worst case, an unbalanced tree degrades to $O(n^2)$ and acts like a linked list.

RECOMMENDATION AND CONCLUSION

This project evaluated three data structures: vector, hash table, and binary search tree for managing course data in an advising system. Each structure was analyzed based on worst case runtime, memory usage, and its ability to meet the program requirements. All three structures have a worst case runtime of $O(n^2)$, however, they differ in how they handle data storage, retrieval, and ordering. The vector is

simple and memory-efficient but performs poorly for repeated searches. The hash table improves lookup performance in average-case scenarios but requires additional memory and does not maintain sorted order. The binary search tree provides a balanced solution by supporting efficient searching and insertion while maintaining data in a sorted order.

The binary search tree is the recommended data structure because it best satisfies both performance expectations and functional requirements. It eliminates the need for additional sorting when displaying courses and provides an effective balance between efficiency and structure, making it the most appropriate choice for the advising system.